

Driver Monitoring System

1. README / Code Execution

To evaluate the Driver Monitoring System, run the `runDMS.py` script from the command line. Please keep in mind that:

- **Prerequisites:** The system has been specifically realized and tested on **Python 3.11.9** using `mediapipe==0.10.20`, as the MediaPipe package exhibits known compatibility issues with newer Python versions. Even if the official documentation ensures support for Python 3.9 - 3.12, it is not guaranteed that the code will work properly using different configurations. The system also requires the `opencv-python`, `numpy`, `scipy`, and `scikit-learn` packages. It is highly recommended to execute the code within a Python Virtual Environment to prevent dependency conflicts and ensure the exact versions are used.
- **Execution & Debug Mode:** The script must be launched using the standard command `python runDMS.py`. It is possible to append the `debug` or `--verbose` keyword at the end of the command (e.g., `python runDMS.py debug`) to show more internal algorithm metrics (Raw HR, FastICA components, buffer size, and gaze ratios, together with timers and cumulative timers for the long and short distractions detection).

Disclaimer: this feature is provided strictly as an extra diagnostic tool and is not intended as the default execution mode.

- **User Controls:** The program can be terminated by pressing the ESC key or by clicking the standard close button (X) on the window.

2. Core Application: `runDMS.py`

The `runDMS.py` is, as requested, the main module of the system. Its primary responsibilities are video acquisition, facial landmark extraction, data distribution to the core modules, and rendering the output.

- **Initialization:** The script initializes the OpenCV `VideoCapture` to read frames from the default webcam, and sets up the `mediapipe.solutions.face_mesh`, extracting the landmarks indices for the main facial element that will be monitored during the execution (eyes, iris and nose).
- **Data Pipeline:** Inside the main loop, each frame is captured, flipped horizontally, and converted to RGB color space for MediaPipe processing. Once the 478 landmarks are extracted, their normalized coordinates are passed to the previously-initialized `DistractionMonitor` and `RPPGMonitor` to estimate focus state and heart-beat rate.

- **User Interface (GUI):**
 - **Mediapipe:** The script draws the eye contours in red, green rhomboids on the irises, and 3 blue points on the nose tip using OpenCV drawing utilities.
 - **Bottom-Right:** Displays the current driver state (e.g., "Focused", "Owl Long distraction"...). The text color changes depending on the state: Green for focused, Orange for distracted, Red for sleep.
 - **Bottom-Left:** Displays the estimated Heart Rate in BPM.
 - **Top-Center:** "WARNING" text appearing whenever a long distraction or sleep condition is triggered by the **DistractionMonitor**.
 - **Top-Left (Debug Mode) :** When executed with the debug flag, a real-time overlay is displayed in the top-left corner detailing the internal mathematical metrics.
 - distraction boolean flags (`is_owl`, `is_lizard`, `eyes_closed`, `is_focused`)
 - raw geometric ratios (`ratio_owl`, `gaze_avg`, `ear`)
 - active timers (`cumul_owl(s)`, `cumul_lizard(s)`, `focus_timer(s)`).
 - `Buffer(s)`
 - actual sampling rate `Fs(Hz)`
 - `Raw HR`
 - FFT peak's `Max Pwr`
 - dominant ICA component `Comp`

3. Distraction Monitoring: **distraction.py**

This module encapsulates the geometric calculations to detect the driver's focus level.

- **Sleep & Microsleep (EAR):** The module calculates the **Eye Aspect Ratio** (EAR) for both eyes. If the EAR falls below a specific threshold (0.22) a continuous timer starts. A duration of **4 seconds** or more triggers "Microsleep", while one of at least **7 seconds** triggers "Sleep".
- **Owl Distraction:** The orientation of the head is estimated by calculating the **ratio** of the distance between the **nose tip** and the **left/right edges of the face**. If the ratio exceeds the normal bounds (0.6 - 1.6; the value is not symmetric because it is the rapport of the left and right distance), the driver is considered to be looking away and the `is_owl` flag is set.
- **Lizard Distraction (Eye Gaze):** The Lizard detection relies strictly on **horizontal distance** calculations. It evaluates the horizontal position of the iris center relative to the inner and outer corners of the eye to determine if the gaze is shifted while the head remains straight (for simplicity, the lizard distraction is not evaluated if the driver is not looking forward). The minimum threshold for the 'Lizard' distraction (gaze deviation (0.4 - 0.6) was estimated empirically: it corresponds approximately to the eye angulation required to look past the edges of a laptop screen, assuming the user is positioned so their face occupies the majority of the vertical frame. For obvious

reasons, this threshold is an approximation and may vary depending on the specific hardware setup.

- **Temporal Logic:**
 - **Long Distractions:** Triggered if the Owl or Lizard conditions are continuously maintained for more than **5 seconds**.
 - **Short Distractions:** The module utilizes a deque structure as a **30-second sliding window** to store the duration of isolated distraction events. If the cumulative distraction time exceeds **10 seconds** within this window, a short distraction warning is triggered.
 - **Reset Condition:** Any active warning is locked until the driver returns to a "Focused" state (head straight, eyes open, gaze forward) for a continuous period of more than **2 seconds**.

***Note:** An implicit hierarchical order is given to the various states. If a "Long Distraction" is triggered (e.g., the driver maintains an Owl state for 5 continuous seconds), the warning state is locked. If, within the remaining time of the 30-second sliding window, the driver performs multiple short glances away that accumulate to more than 10 seconds (which would normally trigger a "Short Distraction"), the system will not downgrade or override the output to "Short Distraction". The output will persistently display the "Long Distraction" warning until the mandatory 2-second continuous focus reset is achieved.*

4. Remote Photoplethysmography: **rPPG.py**

This module implements the complete pipeline for estimating the driver's heart rate directly from the webcam feed.

- **Signal Extraction & Pre-processing:** Following the theoretical pipeline, the module isolates the skin pixels by defining a Region of Interest (ROI) on the driver's **forehead** (MediaPipe landmarks: 10, 9, 67, 297). Cheeks' pixels have also been evaluated but the results were slightly worse so they were excluded from the ROI. For each frame, the spatial **average** of the **RGB** channels is computed and stored in a 10-second rolling buffer. To prepare the data for source separation, the continuous signals undergo a detrending process to remove slow temporal drifts caused by ambient illumination changes, and are then standardized to **zero mean** and unit variance.
- **Independent Component Analysis & FFT:** The core signal separation is performed using `sklearn.decomposition.FastICA`. To maintain strict mathematical coherence with the provided `fastICA.m` implementation, the algorithm is explicitly configured to use the deflation approach (`algorithm='deflation'`) and a cubic non-linearity function (`fun='cube'`, equivalent to MATLAB's `g='pow3'`). Once the three independent components are extracted, a second-order **Butterworth bandpass filter** is applied to strictly isolate plausible human heart rate frequencies (0.8 Hz to 3.0 Hz, corresponding to 48-180 BPM). **Fast Fourier Transform (FFT)** with extensive zero-padding is applied to the filtered components to generate the

power spectrum, in order to identify the dominant frequency peak, theoretically corresponding to the blood volume pulse.

- **Signal Stabilization & Outlier Rejection:** In order to regularize the signal, the raw BPM is initially smoothed across micro-frames using an exponential moving average. Then, the user interface update is deliberately throttled to exactly 1 update per second. At this 1-second interval, the algorithm evaluates the new reading against an **8-slot sliding window (deque)**, which represents the **last 8 seconds** of valid heart rate data. If the newly calculated heart rate spikes by more than **25%** compared to the historical average of the window, it is rejected. If valid, the sliding window shifts to drop the oldest record, and the UI is refreshed with the new moving average.